Week 2 Practice Problems

September 18, 2026

Problem 1 : Little-o and the polynomial-degree fact

Recall the formal definitions (assuming $f(n) > 0, g(n) > 0$ for $n > 0$):

$$O(g(n)) = \{f(n) : \exists c > 0,\ \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0,\ 0 \leq f(n) \leq c\, g(n)\}$$

$$o(g(n)) = \{f(n) : \forall c > 0,\ \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0,\ 0 \leq f(n) < c\, g(n)\}$$

(a) In one or two sentences, explain the key difference between these two definitions. What changes, and what does that change mean intuitively?

(b) Prove directly from the definitions above that little-o is "stronger than" Big-O: that is, if $f(n) = o(g(n))$, then $f(n) = O(g(n))$. (Don't just cite the fact. Instead show it using the $\exists c$ / $\forall c$ structure.)

(c) Let $f(n) = 3n^{25} + 2n^9 + 4$ (the example from lecture). Using the polynomial-degree-k fact a degree-k polynomial with positive leading coefficient satisfies:

(a) $f(n) = o(n^b)$ for $b > k$
(b) $f(n) = \omega(n^b)$ for $b < k$
(c) $f(n) = \Theta(n^b)$ for $b = k$

state which of these three cases applies for $b = 30$, $b = 25$, and $b = 20$.

Solution. (a) Big-O only requires *some* constant c to work ("$\exists c$"), while little-o requires *every* constant c, no matter how small, to work ("$\forall c$"). Intuitively, $f(n) = O(g(n))$ means f is bounded above by *some* multiple of g, whereas $f(n) = o(g(n))$ means f becomes negligible compared to g no matter how tiny a multiple of g you compare it to.

(b) Suppose $f(n) = o(g(n))$. By definition, for every $c > 0$ there exists $n_0 > 0$ such that $f(n) < c\, g(n)$ for all $n \geq n_0$. In particular, this holds for the specific choice $c = 1$: there exists some $n_0 > 0$ such that $f(n) < 1 \cdot g(n)$ for all $n \geq n_0$, hence also $f(n) \leq 1 \cdot g(n)$. This is exactly the statement that $f(n) = O(g(n))$ (using $c = 1$ and that same n_0). So if a function is $o(g(n))$, it must also be $O(g(n))$.

(c) Here $f(n)$ is a degree-$k = 25$ polynomial (leading term $3n^{25}$, positive coefficient). For $b = 30 > 25$: $f(n) = o(n^{30})$. For $b = 25 = 25$: $f(n) = \Theta(n^{25})$. For $b = 20 < 25$: $f(n) = \omega(n^{20})$.

Problem 2 : Logs, exponents, and the story problems

(a) Using the log/exponent identities from lecture

$$\log_b(ac) = \log_b a + \log_b c$$

$$\log_b a^n = n \log_b a$$

simplify $\log_2(16n^3) - 3\log_2(n)$ to a single constant.

(b) Using the change-of-base formula ($\log_b a = \frac{\log_c a}{\log_c b}$), express $\log_8(n)$ in terms of $\log_2(n)$. What does this tell you about how the *base* of a logarithm affects its asymptotic (big-Theta) growth rate?

Solution.

(a) $\log_2(16n^3) - 3\log_2(n) = \log_2 16 + \log_2(n^3) - \log_2(n^3) = \log_2 16 = 4$.

(b) $\log_8 n = \dfrac{\log_2 n}{\log_2 8} = \dfrac{\log_2 n}{3} = \frac{1}{3}\log_2 n$. Since this is just a *constant* $\frac{1}{3}$ times $\log_2 n$, we have $\log_8 n = \Theta(\log_2 n)$. This is why algorithm analysis usually just writes $O(\log n)$ without specifying a base. Changing the base only changes the constant factor, which Θ and O notation ignore.

Problem 3 : Solving a simple recurrence

Consider the recurrence from lecture:

$$T(1) = 1, \qquad T(n) = T(n-1) + 3 \text{ for } n > 1.$$

(a) Compute $T(2)$, $T(3)$, and $T(4)$ directly from the recurrence.

(b) Find a closed-form (non-recursive) expression for $T(n)$, and verify it against your answers to part (a). (Hint: work out the recurrence for a few steps and look to see if you find a pattern).

Solution. (a) $T(2) = T(1) + 3 = 1 + 3 = 4$. $T(3) = T(2) + 3 = 4 + 3 = 7$. $T(4) = T(3) + 3 = 7 + 3 = 10$.

(b) $T(n) = T(n-1)+3 = T(n-2)+3+3 = T(n-3)+3+3+3 = \cdots = T(1)+3(n-1) = 1 + 3(n-1) = 3n - 2$.

Safety check: $T(2) = 3(2) - 2 = 4$. $T(3) = 3(3) - 2 = 7$. $T(4) = 3(4) - 2 = 10$. All match part (a). So $T(n) = 3n - 2$.

Problem 4 : The substitution (induction) method

Consider the recurrence

$$T(1) = 1, \qquad T(n) = 2T(\lfloor n/2 \rfloor) + n^2 \text{ for } n > 1,$$

for n where $n \geq 0$.

(a) Guess an asymptotic upper bound for $T(n)$ of the form $O(n^2)$. Then, following the structure from lecture, state precisely what claim you need to prove by induction (i.e., write down the "there exist $c > 0, n_0 > 0$ such that..." statement).

(b) Carry out the induction step: assuming $T(k) \leq ck^2$ for all integers k satisfying $n_0 \leq k < n$, show $T(n) \leq cn^2$ for an appropriate choice of c. What constraint on c do you need? (You'll need $\lfloor n/2 \rfloor \leq n/2$ at some point to get rid of the floor.)

(c) Carry out the base case for $n_0 = 1$, and state the final values of c and n_0 that make the whole proof work.

Solution. (a) Guess: $T(n) = O(n^2)$. Precisely: we want to show there exist $c > 0, n_0 > 0$ such that $T(n) \leq cn^2$ for all integers $n \geq n_0$.

(b) Inductive hypothesis: $T(k) \leq ck^2$ for all integers k with $n_0 \leq k < n$. Induction step:

$$T(n) = 2T(\lfloor n/2 \rfloor) + n^2 \;\leq\; 2c\lfloor n/2 \rfloor^2 + n^2 \overset{(\star)}{\leq} 2c\left(\frac{n}{2}\right)^2 + n^2 \;=\; \frac{c}{2}n^2 + n^2 \;=\; \left(\frac{c}{2} + 1\right) n^2,$$

where $(\star)$ uses $\lfloor n/2 \rfloor \leq n/2$ (valid since both sides are non-negative, so squaring preserves the inequality). This is precisely the step that gets rid of the floor. We need $\left(\frac{c}{2} + 1\right) n^2 \leq cn^2$, i.e., $c \geq 2$. So the induction step works for any $c \geq 2$, for *every* integer $n \geq 2$ (not just powers of 2), since all we ever used was $\lfloor n/2 \rfloor \geq n_0 = 1$, which holds for every $n \geq 2$.

(c) Take $n_0 = 1$. Base case: need $T(1) \leq c \cdot 1^2 = c$. Since $T(1) = 1$, this holds as long as $c \geq 1$. This already satisfied by $c \geq 2$ from part (b). So $c = 2$, $n_0 = 1$ completes the proof: $T(n) \leq 2n^2$ for all integers $n \geq 1$, i.e., $T(n) = O(n^2)$.

Problem 5 : The asymptotic-notation warning, and comparing methods

(a) Lecture warns against writing asymptotic notation inside an inductive hypothesis, and shows this bad "proof" for $T(n) = 2T(n/2) + n$: Inductive hypothesis: $T(n) = O(n)$. Induction step: $T(n) = 2 \cdot O(n/2) + \Theta(n) = O(n)$. This appears to show $T(n) = O(n)$. However, we know from the recursion-tree method (Lecture 3/4) that the true answer is $T(n) = \Theta(n \log n)$, which is not $O(n)$. What exactly goes wrong with this argument? (Hint: "$T(n/2) = O(n/2)$" just says some constant works for the smaller subproblem. Does the argument ever check that it's the same constant needed for $T(n)$ itself? What happens if you try to verify that directly, with one fixed constant used consistently on both sides?)

(b) In a few sentences, compare the recursion-tree method and the substitution (induction) method as tools for solving a recurrence: what does each one give you, and what's an advantage of each?

Solution. (a) First, observe the final step closely: $2 \cdot O(n/2) + \Theta(n) = O(n)$. To be clear about what's not the issue, the $\Theta(n)$ term (cost of the merge itself) it completely fine as it is genuinely just n, every time, with nothing hidden in it. The issue is entirely in the $O(n/2)$ piece. The phrase $T(n/2) = O(n/2)$ is shorthand for "there is some constant c with $T(n/2) \leq c \cdot (n/2)$." To conclude $T(n) = O(n)$, we'd need a constant with $T(n) \leq c \cdot n$. The argument never actually

checks whether the constant that works for the smaller subproblem also works for $T(n)$ itself.

Let's check directly whether a single fixed constant can actually work here. Suppose $T(n/2) \leq c \cdot (n/2)$ for some constant c:

$$T(n) = 2T(n/2) + n \;\leq\; 2\left(c \cdot \frac{n}{2}\right) + n \;=\; cn + n.$$

For $T(n) \leq cn$ to follow from this, we'd need $cn + n \leq cn$, i.e., $n \leq 0$. That's false for every $n > 0$, no matter which c we started with.

That is exactly the issue here. $O(\cdot)$-notation asserts that some constant exists, but it doesn't come with a built-in guarantee that the same constant survives the substitution step of the recurrence. The moment you fix one actual constant and track it through the algebra, as above, the induction step fails. This is exactly why $T(n) = O(n)$ turns out to be false, despite what the argument seemed to show.

(b) The recursion-tree method computes a closed-form expression for $T(n)$ directly, by summing the cost at each level of recursion. The substitution (induction) method instead starts from a guessed asymptotic bound and proves that bound by induction. One practical difference is that the recursion tree doesn't require guessing the answer first, while the substitution method does.